

PRÉPAS SCIENTIFIQUES

**MP/MP* - PSI/PSI*
PT/PT* - MPI/MPI***

**CONFORME AU
NOUVEAU
PROGRAMME**

Alain Caignot • Vincent Crespel • David Violeau

Sciences industrielles de l'ingénieur

TOUT-EN-UN

COURS • MÉTHODES • ENTRAÎNEMENTS ET SUJETS • CORRIGÉS



Tout le cours avec :

- Les objectifs-clés du programme
- Les notions à maîtriser
- Plus de 700 illustrations, schémas et diagrammes explicatifs



Entraînement intensif avec :

Des exercices d'application et d'approfondissement inédits, adaptés ou issus de sujets de concours



Des outils pour réviser :

- Les synthèses de cours
- Les méthodes et techniques



Tous les corrigés détaillés



OFFERT EN LIGNE

- ▶ **100 flashcards** interactives
- ▶ Toutes les **synthèses** de cours à télécharger

Vuibert

Vrai ou Faux

	Vrai	Faux
a) L'Intelligence Artificielle permet de reproduire le comportement du vivant de manière autonome.	☐	☐
b) L'apprentissage automatique consiste à réaliser des prédictions à partir de beaucoup de données d'entrée.	☐	☐
c) Il faut uniquement des données d'entrée pour qu'un algorithme supervisé puisse organiser les données en groupe ou prévoir des valeurs de sortie.	☐	☐
d) Un algorithme de classification consiste à classer les données d'entrée selon leur valeur.	☐	☐
e) La phase d'inférence correspond à l'étape où l'on prédit des sorties pour de nouvelles entrées sur le modèle obtenu après la phase d'apprentissage.	☐	☐
f) La régression linéaire multivariée consiste à déterminer une valeur de sortie comme étant la somme pondérée de plusieurs attributs d'entrée.	☐	☐
g) L'algorithme k-NN permet de traiter des problèmes de classification uniquement.	☐	☐
h) Les réseaux de neurones déterminent autant de poids et de biais qu'il y a de neurones dans le modèle.	☐	☐

Exercices d'application



EXERCICE 1 Type d'apprentissage

Pour chacun des exemples décrits ci-après, indiquer si le problème est de type supervisé ou non supervisé et si c'est un problème de classification ou de régression.

- 1) On cherche à déterminer à partir d'images de fleurs dans quelle famille elles appartiennent. On dispose pour cela de plusieurs images dont on connaît la famille d'appartenance.
- 2) À partir de données telles que les habitudes d'achat, la localisation géographique, les habitudes de déplacement d'un titulaire de carte, ainsi que des données en temps réel sur l'utilisation des cartes telles que ce qu'ils essaient d'acheter, où ils essaient de l'acheter et ce qu'ils ont acheté le même jour, des fabricants de cartes bancaires souhaitent repérer si une transaction est frauduleuse ou non.
- 3) Un algorithme trie automatiquement les photos en fonction des personnes qui les constituent.
- 4) À partir de données de pluviométrie, de température, de type de sol associé à un rendement obtenu pour des parcelles agricoles, on cherche à prévoir d'une parcelle agricole quelconque.
- 5) En fonction de sa taille, on cherche à déterminer si une tumeur est bénigne ou non.



EXERCICE 2 Choix d'algorithmes d'apprentissage automatique

Pour chacun des exemples proposés ci-après, indiquer quelles méthodes d'apprentissage il est possible d'utiliser pour résoudre le problème. Préciser la nature des données d'entrée et de sortie.

- 1) On dispose d'enregistrements sonores et les textes associés. On souhaite déterminer le texte correspondant à un nouvel enregistrement sonore.
- 2) On dispose de données d'angles d'un robot 2 axes et les couples des 2 moteurs correspondant. On cherche à déterminer les couples pour d'autres positions.
- 3) On dispose de données concernant des liquides : leur couleur, leur viscosité et leur saturation. On souhaite déterminer leur type.
- 4) On dispose de données concernant des patients atteints de diabète, leur âge, leur sexe, l'indice de masse corporelle, la moyenne de la pression dans le sang et 6 autres données relatives au sang. L'objectif est de prédire le taux d'évolution de la maladie après une année.
- 5) On cherche à trier des fruits selon leur forme (elliptique ou sphérique), leur poids et leur texture (rugueux ou lisse).

EXERCICE 3

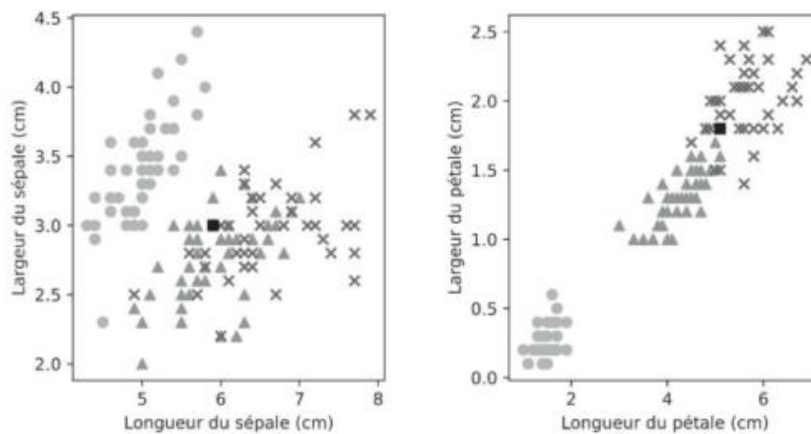
Classification d'Iris par la méthode k-NN

Un exemple très souvent trouvé sur internet concerne la classification d'Iris à partir de mesures réalisées par un botaniste, Ronald Fisher en 1936. Il a déterminé pour 150 fleurs la largeur et la longueur des pétales et des sépales en cm et relevé les espèces de chaque iris parmi : setosa, versicolor et virginica.



► Figure 7.15. Variétés d'Iris et grandeurs mesurées.

Les données sont disponibles dans scikit-learn. On peut alors tracer la répartition des iris en fonction des caractéristiques mesurées. On obtient les deux graphiques de la figure 7.16. Les ronds correspondent aux variétés setosa, les triangles aux variétés versicolor et les croix aux variétés virginica.



► Figure 7.16. Variétés d'Iris et grandeurs mesurées.

- 1) Si on utilise les mesures basées sur les pétales, à quelle catégorie appartient la fleur marquée par un carré?

L'identification est plus délicate si on utilise les mesures sur les sépales. On décide d'utiliser un algorithme k-NN pour une classification automatique. On utilise comme critère la distance entre le point considéré et les autres points en se basant uniquement sur les coordonnées correspondantes aux longueurs et largeurs de sépale.

- 2) Indiquer alors quelle est la variété la plus proche du point carré.
- 3) Si on considère 5 voisins, quel est alors le résultat que donnera l'algorithme?

EXERCICE 4

Régression linéaire multivariée

On récupère un tableau T contenant des données de sexe (masculin/féminin) noté s , âge a , poids m , fumeur ou non f , pression sanguine (notée p). On dispose de 100 échantillons. L'objectif est de déterminer une relation entre la pression sanguine et les autres paramètres.

- 1) Comment doit-on mettre en forme le tableau de données pour pouvoir utiliser les modules permettant de réaliser une régression linéaire?
- 2) Quelle équation entre les variables obtiendra-t-on en réalisant une régression linéaire multivariée?
- 3) Après utilisation de l'algorithme `LinearRegression`, on obtient un coefficient de corrélation de 0,51. Que peut-on en conclure? Que pourrait-on faire pour améliorer cette corrélation.

EXERCICE 5 Réseaux de neurones

On considère un réseau de neurones sans couche cachée pour résoudre un problème constitué de 6 attributs et de 3 grandeurs de sortie à prédire qui peuvent prendre n'importe quelles valeurs réelles entre 0 et 1. On dispose de 1000 échantillons pour lesquels on connaît les valeurs de sortie.

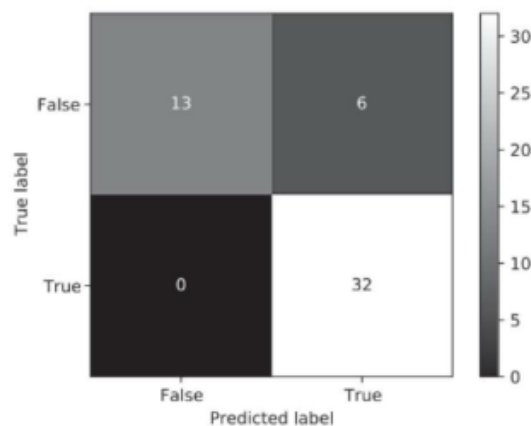
- 1) Indiquer s'il s'agit d'un problème de classification ou de régression.
- 2) Combien de neurones faut-il utiliser?
- 3) Préciser le nombre d'inconnues de poids et biais nécessaires pour résoudre ce problème.
- 4) Quel type de fonction d'activation est pertinente compte-tenu des sorties souhaitées?

EXERCICE 6 Analyse de l'ionosphère

Des données radar ont été recueillies par un réseau phasé de 16 antennes haute fréquence. Les cibles étaient des électrons libres dans l'ionosphère. Les « bons » retours radar sont ceux qui mettent en évidence un certain type de structure dans l'ionosphère. Les « mauvais » retours sont ceux qui ne le font pas, les signaux correspondant traversent l'ionosphère. On dispose donc de 350×32 signaux radar, valeurs réelles comprises entre -1 et 1 . Pour chacun des 350 échantillons, une étiquette « bon » (1) ou « mauvais » (0) est associée.

On cherche à mettre en place un modèle permettant de prédire pour un échantillon de 32 signaux radar si le retour est bon ou mauvais.

- 1) Quel type de régression doit-on mettre en place pour créer ce modèle?
- 2) Que faut-il faire pour analyser la pertinence de l'apprentissage mis en place?
- 3) On obtient la matrice de confusion suivante sur 50 échantillons. Quelles informations peut-on en tirer? Calculer la justesse associée à cette prédiction.



► Figure 7.17. Matrice de confusion pour 50 échantillons.

Exercices d'approfondissement



EXERCICE A Asservissement par traitement d'images

Pour piloter une plateforme dite hexapode, une solution évoluée consiste à estimer les positions réelles des extrémités des tiges des vérins en utilisant une caméra et un algorithme de traitement d'images.

Le positionnement de la plateforme mobile est déterminé à l'aide d'images fixes capturées par plusieurs caméras placées autour de la plateforme Hexapode (voir figure 7.18). Par souci de simplification, on ne s'intéressera qu'au traitement des images d'une seule de ces caméras.



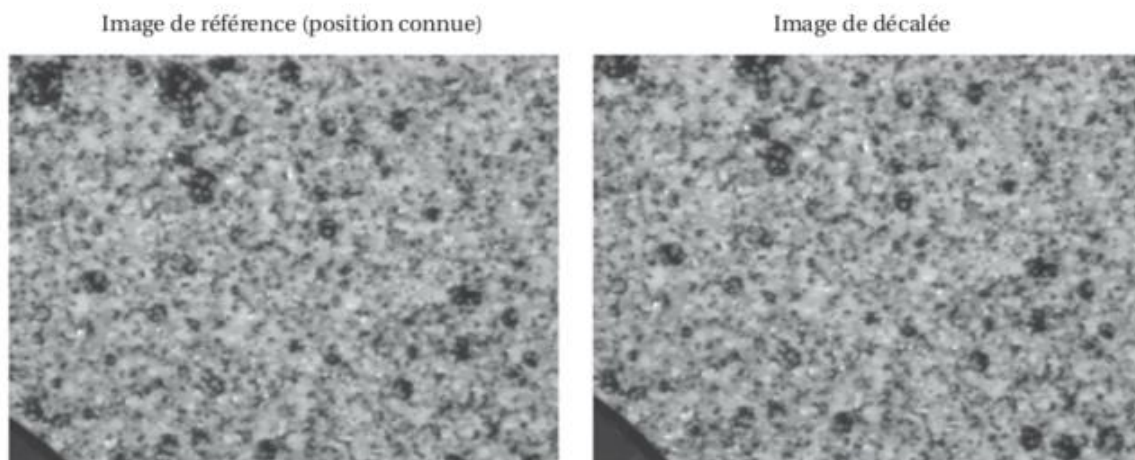
► Figure 7.18. Système d'asservissement en position d'un Hexapode par traitement d'images.

Une image peut être représentée sous la forme d'un tableau de valeurs, chaque case correspondant à un pixel de l'image ; les caméras étant monochromatiques, chaque valeur du tableau représente le niveau de gris codé sur 8 bits du pixel correspondant (pour information : 0 = noir, 255 = blanc).

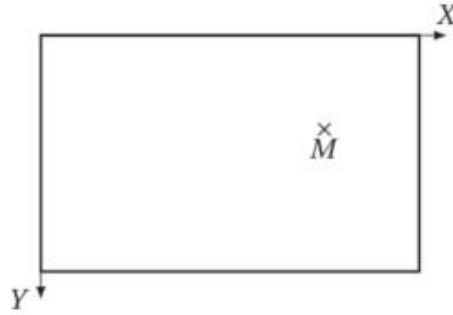
Pour simplifier le problème, on admet que chaque image peut être représentée par une fonction continue des variables continues de l'espace 2D :

- $f(X, Y)$ désigne l'image de référence, associée à la plateforme en position nominale supposée connue (comme illustré à titre d'exemple figure 7.19) ;
- $g(X, Y)$ désigne une image prise à un instant donné (voir figure 7.19) ;

où X et Y désignent respectivement l'abscisse et l'ordonnée d'un point M de l'image (voir figure 7.20).



► Figure 7.19. Images en position nominale et décalée.



► Figure 7.20. Repère image.

L'objet de l'algorithme de traitement d'images est de trouver le déplacement $\vec{U}(X, Y)$ de composantes $U_X(X, Y)$ et $U_Y(X, Y)$ dans le repère de l'image, qui permet de relier l'image de référence à l'image actuelle : $g(X + U_X(X, Y), Y + U_Y(X, Y)) = f(X, Y) + n(X, Y)$ où $n(X, Y)$ désigne un « bruit de mesure » (ce terme modélise, entre autres, des erreurs dans la prise d'images et dans la discrétisation des signaux).

Le principe de l'algorithme est similaire à celui utilisé dans les algorithmes d'apprentissage automatique. Il consiste à minimiser la fonction « coût » suivante :

$$J(U_X(X, Y), U_Y(X, Y)) = \iint n(X, Y)^2 dX dY$$

- 1) Interpréter physiquement sur un plan qualitatif ce principe de minimisation.

Afin de résoudre ce problème, on cherche une solution de la forme :

$$\vec{U}(X, Y) = \sum_{i=1}^6 \alpha_i \vec{\varphi}_i(X, Y)$$

où les vecteurs $\vec{\varphi}_i(X, Y)$ désignent les six mouvements élémentaires de corps rigide (3 translations élémentaires, 3 rotations élémentaires, supposées connues) de la plateforme S par rapport au support S_0 .

- 2) En introduisant dans l'expression de la fonction coût le développement en série de Taylor à l'ordre 1 de $g(X + U_X(X, Y), Y + U_Y(X, Y))$ en (X, Y) , donner l'expression de la fonction coût en fonction de $f(X, Y)$, $g(X, Y)$, $\vec{\nabla} g(X, Y)$, α_i et $\vec{\varphi}_i(X, Y)$ où $\vec{\nabla} g$ désigne le gradient spatial de g .
- 3) Écrire les équations vérifiées par les coefficients α_i qui correspondent à la minimisation de la fonction coût obtenue à la question précédente et qui permettent ainsi d'obtenir l'expression du déplacement mesuré.
- 4) Montrer que la détermination des coefficients α_i peut être obtenue par la résolution d'un système linéaire de la forme :

$$\begin{pmatrix} R_{11} & \cdots & R_{16} \\ \vdots & \ddots & \vdots \\ R_{61} & \cdots & R_{66} \end{pmatrix} \begin{pmatrix} \alpha_1 \\ \vdots \\ \alpha_6 \end{pmatrix} = \begin{pmatrix} \iint (f(X, Y) - g(X, Y)) \vec{\varphi}_1(X, Y) \cdot \vec{\nabla} g(X, Y) dX dY \\ \vdots \\ \iint (f(X, Y) - g(X, Y)) \vec{\varphi}_6(X, Y) \cdot \vec{\nabla} g(X, Y) dX dY \end{pmatrix}$$

et donner l'expression générique du terme R_{ij} .

- 5) En vue d'une implantation numérique, proposer une méthode d'approximation numérique du gradient $\vec{\nabla} g(X, Y)$

Cette étude montre que l'on peut formuler le problème de la détermination du positionnement de la plateforme mobile comme la résolution d'un système d'équations linéaires.

EXERCICE B Mesures sur un moteur thermique

On étudie un moteur thermique pour lequel un relevé du taux de carburant consommé et de la vitesse de rotation est effectué. On relève également le couple fourni par le moteur et l'émission d'oxyde nitreux. L'objectif est de prévoir le couple et l'émission d'oxyde nitreux pour toutes les valeurs de vitesse et de taux de carburant comprises entre les valeurs extrêmes relevées expérimentalement.

Le fichier `inputs.csv` contient sur la première ligne les valeurs du taux de carburant et sur la seconde ligne les valeurs de vitesse. Les valeurs sont séparées par des points-virgules. Le fichier `inputs.csv` contient sur la première ligne les couples pour chaque mesure et sur la deuxième ligne l'émission d'oxyde nitreux (valeurs séparées par des virgules).

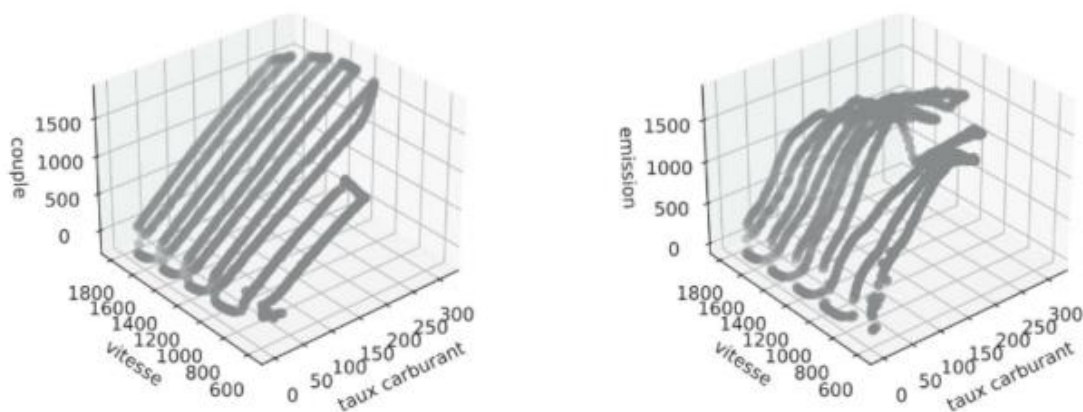
- 1) Proposer un script permettant de lire les fichiers et de créer des variables X contenant en colonne le taux de carburant et la vitesse et la variable Y contenant en colonne le couple et pour la deuxième colonne l'émission d'oxyde nitreux. On pourra utiliser les tableaux Python `numpy` si nécessaire pour stocker X et Y .
- 2) Compte-tenu de l'objectif souhaité et des données utilisées, indiquer de quel type de problème d'apprentissage il s'agit.

On trace les différents points dans un diagramme tridimensionnel. On obtient les courbes de la figure 7.21.

On utilise le code suivant pour estimer les valeurs de couple.

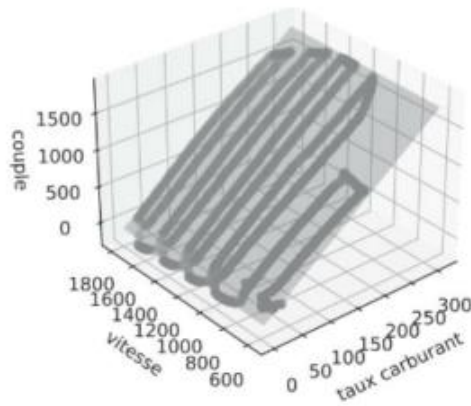
```
from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(X,Y[0])
print(model.score(X,Y[0]))
```

- 3) Expliquer ce qui est fait avec ces lignes de code et pourquoi un tel apprentissage est pertinent. La dernière ligne renvoie 0,994.
- 4) On cherche maintenant à prédire les valeurs de couple pour tous les points situés entre les valeurs $[0,314]$ pour le taux de carburant et $[576,1800]$ pour la vitesse. Définir un nouveau tableau X_p de dimension $(100 \times 100, 2)$ contenant l'ensemble des points pour lesquels on souhaite prédire la valeur du couple.



► Figure 7.21. Représentation des données.

La commande `Yp=model.predict(Xp)` permet d'obtenir les données représentées en continu figure 7.22. La commande `print(model.coef_)` fournit les coefficients de l'approximation et renvoie environ `[6.0 0.034]`, la commande `print(model.intercept_)` renvoie `-136,83` qui correspond au biais.



► **Figure 7.22.** Représentation des points de mesure et de l'approximation réalisée.

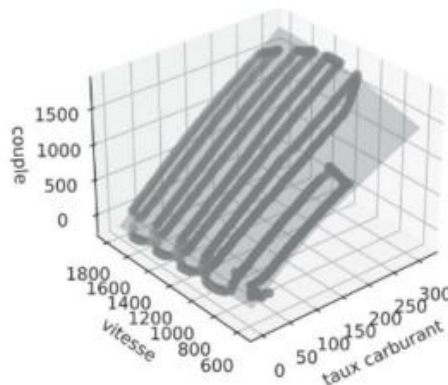
- 5) Que peut-on dire du modèle ainsi défini? Indiquer l'équation numérique obtenue entre le couple, le taux de carburant et la vitesse.

Le même modèle appliqué sur $Y[1]$ donne un score de 0.55 ce qui n'est pas satisfaisant. On décide alors de mettre en place une autre méthode d'apprentissage basée sur les réseaux de neurones. On utilise les instructions suivantes avec $n1$ et $n2$ le nombre de neurones des couches cachées 1 et 2.

```
from sklearn.neural_network import MLPRegressor
n1,n2=100,50
clf = MLPRegressor( hidden_layer_sizes=(n1,n2),max_iter=1000)
clf.fit(X, Y[0])
print(clf.score(X,Y[0]))
```

On remarque qu'en mettant un nombre de neurones trop petit pour chacune des couches cachées, l'algorithme peine à converger pour 1000 itérations. On retient donc $n1 = 100$ et $n2 = 50$ après plusieurs essais. On obtient un score de 0,996 et les points obtenus sont tracés sur la figure 7.23.

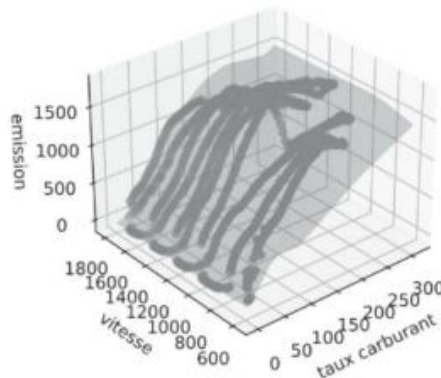
- 6) Comparer l'approximation obtenue par réseau de neurones avec celle obtenue par régression linéaire. Quels sont les avantages et inconvénients d'utiliser cette méthode d'approximation? Indiquer notamment le nombre de poids et biais utilisés dans le calcul compte-tenu des neurones retenus pour les couches cachées et des données d'entrée et de sortie.



► **Figure 7.23.** Représentation des points de mesure et de l'approximation réalisée par réseau de neurones.

Remarque

En appliquant cette méthode pour l'approximation de l'émission d'oxydes nitreux, en adaptant le nombre de neurones pour les couches cachées, on arrive à monter à un score de 0,77. Cependant lorsqu'on regarde l'approximation obtenue sur toute la zone d'intérêt, on observe que les différences sont grandes. Il ne semble donc pas possible de trouver une bonne approximation de l'émission d'oxydes nitreux sur toute la zone d'intérêt à partir des seules données de taux de carburant et de vitesse.



► Figure 7.24. Représentation des points de mesure et de l'approximation réalisée par réseau de neurones sur l'émission d'oxydes nitreux.



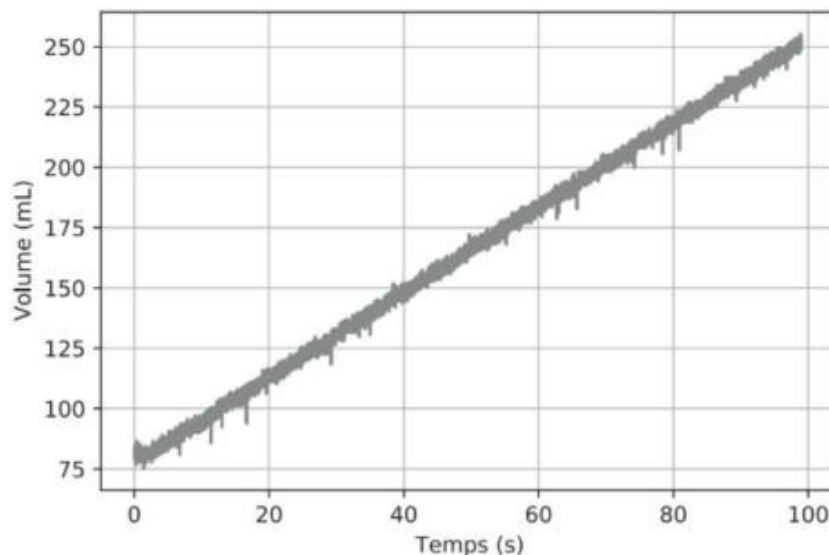
EXERCICE C

Chaîne de pesage d'un automate de prélèvement sanguin

On s'intéresse à nouveau au système de prélèvement de sang étudié dans un exercice du chapitre 5. Un capteur de force permet de mesurer en temps réel le sang contenu dans une poche posée sur le système. Le signal brut est filtré pour pouvoir ensuite être analysé. L'objectif de la mesure est de déterminer avec précision une masse donnée de sang lors d'un prélèvement, de manière reproductible.

On donne figure 7.25 l'évolution du volume (en mL) de la poche de sang lors d'un prélèvement. On dispose d'un vecteur de valeurs temporelles T et d'un vecteur de données filtrées V .

- 1) Quelle approximation peut-on faire compte-tenu de forme de la courbe ? Donner une expression approchée de V en fonction du temps T .



► Figure 7.25. Évolution du volume de sang lors d'un prélèvement.

On importe en Python la fonction `linregress` : `from scipy.stats import linregress`. L'exécution de la commande `sol = linregress(T,V)` donne les informations suivantes :

```
sol = linregress(T,V)
print(sol)
>>> LinregressResult(slope=1.757, intercept=77.736, rvalue=0.9994)
```

- 2) Rappeler en quelques lignes le principe de la détermination des coefficients intervenant dans ce résultat.

L'inconvénient de l'approximation réalisée précédemment est qu'elle est faite à la fin de la mesure. Comme on veut savoir quand arrêter le prélèvement, il faut donc réaliser cette approximation en continu.

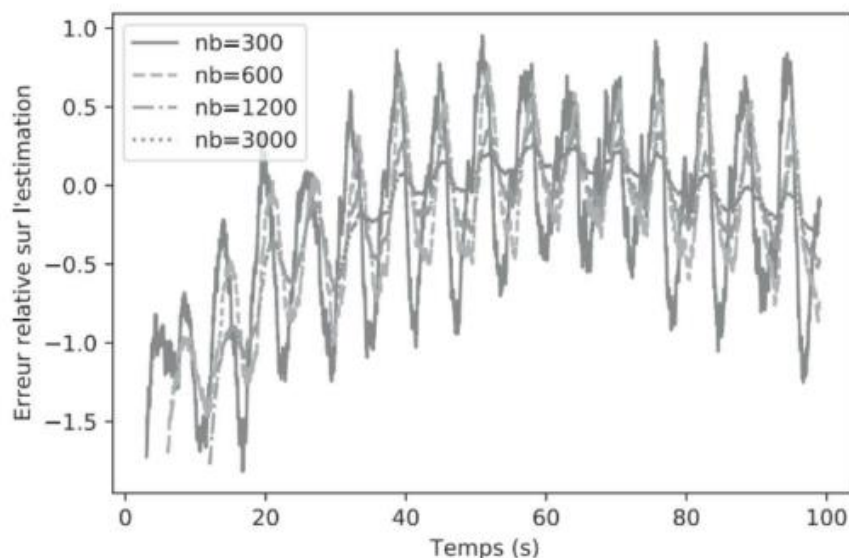
Il serait possible de réaliser l'approximation sur l'ensemble des données mais, afin que le temps de calculer l'approximation soit constant au cours du prélèvement, on réalise une régression linéaire mobile sur les N derniers points de mesure.

On considère N points consécutifs pour lesquels on applique la régression linéaire. On détermine les 2 coefficients notés a et b par l'intermédiaire de la méthode `linregress` et on calcule alors l'approximation évaluée au temps courant. Ceci est fait uniquement lorsqu'il y a plus de N points précédents.

- 3) Proposer une fonction `linregmobile(V,T,v,t,N)` qui prend en argument la liste `v` des mesures de volume, la liste `T` des temps et l'entier N correspondant au nombre d'échantillons à considérer pour faire la régression mobile. La fonction renvoie la valeur approchée du volume pour le dernier temps de mesure. On réutilisera la fonction `linregress`.

On trace ensuite la différence entre l'approximation globale faite au départ et les courbes approchées obtenues par régression linéaire mobile. On obtient les courbes de la figure 7.26 pour différents échantillons N .

- 4) Indiquer l'erreur maximale commise et conclure sur le nombre de points à utiliser pour faire cette régression linéaire mobile.



► Figure 7.26. Évolution de l'erreur d'approximation.

EXERCICE D Détection automatique de chiffres manuscrits

L'objectif est de détecter la valeur d'un chiffre à partir d'une image manuscrite de celui-ci. Pour déterminer automatiquement un chiffre, il faut posséder un grand nombre d'images pour lesquelles le chiffre est connu. Le module `sklearn` contient de nombreuses images en niveaux de gris référencées de chiffres.



► Figure 7.27. Exemples d'images de chiffres contenues dans la librairie `sklearn`.

On charge dans Python les différentes images et on exécute les commandes suivantes :

```
from sklearn.datasets import load_digits
digits = load_digits()
print(digits.data.shape)
print(digits.target.shape)
```

Ce script renvoie les valeurs suivantes :

```
>>> (1797, 64)
>>> (1797,)
```

- 1) Indiquer le nombre d'images disponibles dans le module. Combien d'attributs sont utilisés pour chaque image ?
- 2) Préciser quel type d'apprentissage il va falloir mettre en place. Proposer des méthodes, sans les détailler, permettant de résoudre le problème posé.

Les images en niveaux de gris sont visualisables avec la commande `plt.imshow(digits['images'][i], cmap = 'Greys_r')` du module `matplotlib` où `i` est le numéro de l'image d'intérêt. On obtient, par exemple, les images de la figure 7.28 correspondant au chiffre 4.



► Figure 7.28. Représentation des images pour $i = 4$ à gauche et $i = 100$ à droite.

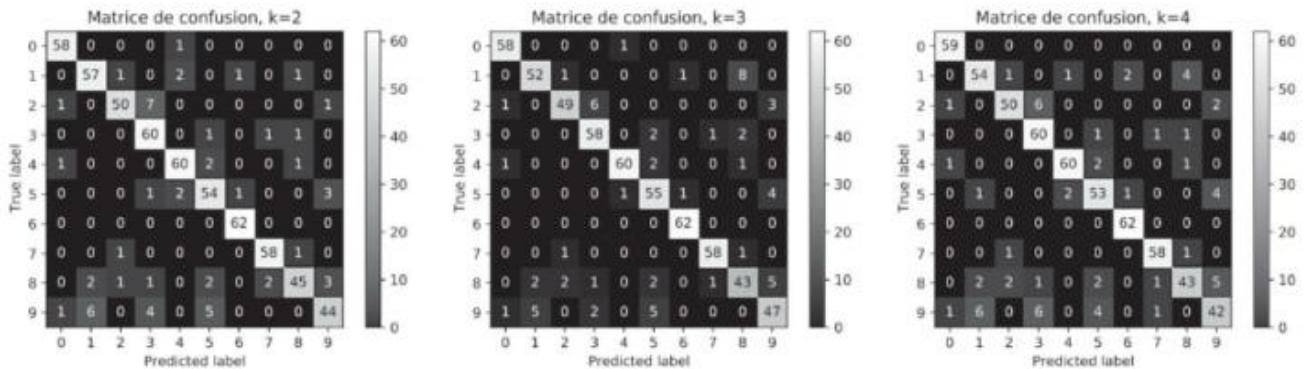
- 3) Indiquer à quoi correspondent les 64 attributs utilisés.

On utilise le code suivant pour réaliser l'apprentissage automatique.

```
from sklearn.neighbors import KNeighborsClassifier
n=100
X = digits.data[:n,:]
y = digits.target[:n]
model = KNeighborsClassifier(n_neighbors=5)
model.fit(X, y)
```

- 4) Expliquer ce qui est fait à travers ces quelques lignes de code.
- 5) La commande `model.score(X,y)` renvoie la valeur de la justesse du modèle. On réalise les commandes précédentes pour n inclus dans [20 50 100 200 500 1000] et on affiche la justesse à chaque fois. On obtient les résultats suivants [0,6 0,94 0,98 0,985 0,992 0,991]. Quelles conclusions peut-on obtenir de cette analyse?

On réalise ensuite des prédictions sur des images comprises entre les indices 1087 et 1693 en testant différentes valeurs de k . On affiche la matrice de confusion pour chaque valeur de k . On obtient les résultats de la figure 7.29.



► Figure 7.29. Matrice de confusion obtenue pour différentes valeurs k .

- 6) Indiquer combien d'images correspondant aux 1 ont été traitées et combien ont été correctement prédites dans le cas $k = 3$? Indiquer également combien d'images ont été identifiées comme des 1 alors qu'elles ne l'étaient pas.
- 7) Commenter l'influence de k sur les résultats obtenus.

EXERCICE E Détection des mouvements humains

On utilise un smartphone pour détecter les mouvements humains. Le smartphone est placé d'une certaine manière dans la poche de pantalon de l'utilisateur et les données brutes d'accélération selon les 3 directions sont relevées lors d'un mouvement particulier de l'utilisateur. À partir de ces mesures, 60 caractéristiques sont calculées. Ces caractéristiques permettent d'extraire les effets haute fréquence ou basse fréquence. On calcule par exemple la moyenne des accélérations non filtrées, la valeur moyenne quadratique (rms), la valeur crête, maximale des signaux filtrés mais également des grandeurs fréquentielles (après transformation de Fourier) telle que la valeur de l'amplitude du pic fréquentiel, la fréquence correspondante, la bande passante, la fréquence moyenne, etc.

Le *dataset* utilisé contient 24075 enregistrements de 60 caractéristiques. Pour chaque enregistrement, le mouvement de l'utilisateur a été relevé parmi la liste ['Assis', 'Debout', 'Marche', 'Course', 'Danse']. L'objectif est de déterminer le mouvement en fonction d'un nouvel enregistrement d'accélération (et du calcul des 60 caractéristiques).

- 1) Expliquer quel type de problème est traité ici et quel algorithme d'IA il est possible d'utiliser.
- 2) Définir la matrice des données d'entrée notée X (dimensions et ce qu'elle contient) ainsi que les données de sortie notées y (dimensions, valeurs) pour pouvoir utiliser les algorithmes de scikit-learn.

Les données sont lues à partir d'un fichier qui contient en première ligne les noms des grandeurs mesurées séparées par des points-virgules et les données pour chaque enregistrement sur les lignes suivantes (données de type flottant séparées par des points-virgules également). Des méthodes permettent de déterminer les attributs qui sont les plus pertinents dans la classification à réaliser. On se limite ainsi uniquement aux colonnes d'indice 1 à 6 dans les données lues.

- 3) Écrire un script permettant de lire le fichier `data.csv` et de créer la matrice X . On ne sélectionnera que les colonnes d'indice 1 à 6.
- 4) Expliquer ce qui est fait dans le code suivant :

```

import numpy as np
nb=int(0.1*X.shape[0])
inds=np.random.randint(0,24075,nb)
Xa,Xt,ya,yt=[],[],[],[]
for i in range(X.shape[0]):
    if i not in inds:
        Xa.append(X[i,:])
        ya.append(y[i])
    else:
        Xt.append(X[i,:])
        yt.append(y[i])

```

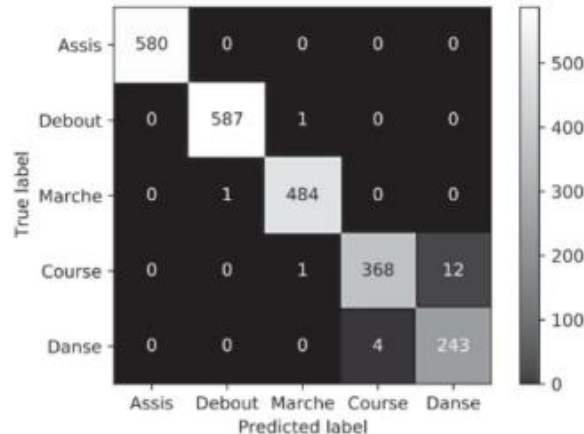
On utilise ensuite une méthode de type k-NN pour réaliser les prédictions avec les instructions suivantes.

```

from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
from sklearn.neighbors import KNeighborsClassifier
import matplotlib.pyplot as plt
labels=['Assis','Debout','Marche','Course','Danse']
model = KNeighborsClassifier(n_neighbors=5)
model.fit(Xa, ya)
print(model.score(Xa,ya))
yp=model.predict(Xt)
cm = confusion_matrix(yt, yp)
disp = ConfusionMatrixDisplay(confusion_matrix=cm,display_labels=labels)
disp.plot()
plt.show()

```

On obtient une valeur d'erreur de 0,995 et la matrice de confusion de la figure 7.30.



► **Figure 7.30.** Matrice de confusion pour la prédiction de mouvements.

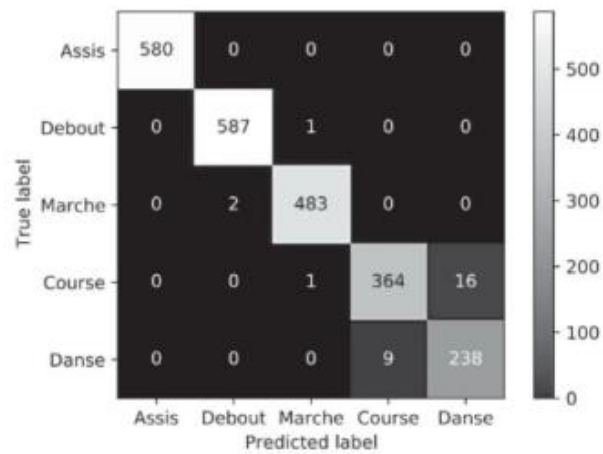
5) Commenter les résultats obtenus et préciser la justesse de la prédiction.

On teste pour finir la méthode des réseaux de neurones en choisissant un réseau à 2 couches cachées de n_1 et n_2 neurones chacune. On calcule l'erreur pour chaque essai en faisant varier n_1 et n_2 .

On obtient les résultats suivants :

- $n_1 = 10$, $n_2 = 2$, erreur de 0,973
- $n_1 = 50$, $n_2 = 10$, erreur de 0,987
- $n_1 = 100$, $n_2 = 10$, erreur de 0,989
- $n_1 = 100$, $n_2 = 50$, erreur de 0,991

Pour le dernier cas, on obtient la matrice de confusion de la figure 7.31.



► **Figure 7.31.** Matrice de confusion pour la prédiction de mouvements par la méthode des réseaux de neurones.

- 6) Indiquer le nombre de poids et biais à déterminer en fonction de n_1 et n_2 .
- 7) Commenter les résultats obtenus et conclure sur la meilleure méthode pour répondre au problème initial.

CORRIGÉS

Corrigés des Vrai/Faux

- a) Faux. L'Intelligence Artificielle est dite « Faible » car les problèmes traités sont spécifiques et l'utilisateur doit intervenir pour paramétrer l'IA.
- b) Vrai. En se basant sur des motifs, des statistiques sur les données d'entrée, les algorithmes d'apprentissage automatique vont permettre de prévoir les valeurs de données de sortie.
- c) Faux. C'est le principe des algorithmes non supervisés. Pour les algorithmes supervisés, l'IA apprend à partir de couples entrée-sortie pour pouvoir ensuite faire des prédictions à partir de nouvelles données d'entrée.
- d) Faux. Il s'agit de prévoir la valeur de données de sortie parmi un ensemble fini de valeurs à partir de données d'entrée quelconques.
- e) Vrai. La phase d'apprentissage consiste à entraîner la machine à partir de couples de données d'entrée-sortie. La phase d'inférence consiste à utiliser le modèle : pour vérifier s'il donne de bon résultat sur des données d'entrée-sortie connus (jeu de test) ou sur de nouvelles entrées dont la sortie est inconnue.
- f) Vrai. Si on note x_i les valeurs des attributs pour un échantillon donné, alors $y = \sum_i a_i x_i + \beta$ où β est appelé le biais.
- g) Faux. En prenant la valeur du groupe majoritaire parmi les k voisins les plus proches, on traite effectivement un problème de classification. Mais en prenant la valeur moyenne des k plus proches voisins, on peut résoudre un problème de régression.
- h) Faux. Il faut effectivement déterminer autant de biais que de neurones (en comptant la couche de sortie) mais il faut déterminer les poids entre les neurones de deux couches successives ce qui augmente beaucoup le nombre d'inconnues de ces problèmes.

Corrigés des exercices d'application

EXERCICE 1

- 1) Dans le premier exemple, étant donné qu'on dispose d'images dont on connaît la famille d'appartenance, le problème est de type supervisé. Le nombre de familles étant fini, c'est un problème de classification.
- 2) Étant donné qu'on dispose de peu d'informations concernant les transactions frauduleuses, il est plus judicieux de traiter le problème de manière non supervisée. C'est un problème de classification binaire.
- 3) C'est à nouveau un problème d'apprentissage non supervisé car on n'indique pas à la machine les personnes présentes sur les images, c'est elle qui gère le classement des images en fonction de caractéristiques similaires. Le nombre de catégories étant fini, c'est un problème de classification.
- 4) On dispose de données sur les rendements précédents en fonction des données d'entrée, l'apprentissage est donc supervisé. Étant donné qu'on souhaite connaître un rendement qui peut prendre n'importe quelle valeur, le problème est un problème de régression.
- 5) On dispose nécessairement de données sur les précédents cancers, l'apprentissage est supervisé et c'est un problème de classification binaire.

EXERCICE 2

- 1) Les entrées sont sûrement des données continues pour chaque enregistrement sonore (fonction du temps). Le nombre de textes (chaînes de caractères) étant limité (mais pouvant être important), on a affaire à un problème de classification multiclasse. On peut utiliser la méthode k-NN ou un réseau de neurones.
- 2) Les positions sont des valeurs réelles et les couples aussi. On a donc affaire à un problème de régression. On peut utiliser des méthodes de régression linéaire multivariée ou polynomiale, on peut également utiliser k-NN ou un réseau de neurones pour résoudre ce problème.
- 3) Les données d'entrées sont soit des valeurs discrètes (couleur), soit des valeurs réelles. La sortie est une valeur binaire. On a donc un problème de classification binaire à résoudre. On peut utiliser une régression logistique ou k-NN ou un réseau de neurones.
- 4) Les données d'entrées sont des nombres pour l'âge ou l'indice de masse corporelle ou les données relatives au sang et une variable binaire pour le sexe. L'évolution de la maladie étant un taux, c'est un nombre réel. On a donc affaire à un problème de régression multivariée qui peut être traité à nouveau par k-NN ou réseau de neurones ou régression directe.
- 5) La forme des fruits est une grandeur binaire, de même que la texture, le poids est un nombre quelconque. Ainsi le problème est un problème de classification binaire à traiter avec une régression logistique ou k-NN ou un réseau de neurones.

EXERCICE 3

- 1) En se basant sur le critère des pétales, on peut dire sans hésiter que la fleur représentée par le carré appartient au groupe des croix soit la variété virginica.
- 2) Le plus proche voisin sur le critère des sépales est une croix donc à nouveau virginica.
- 3) Si on compte les 5 plus proches voisins, on remarque qu'il y a 2 croix et 3 triangles. L'algorithme k-NN renverra donc la catégorie versicolor. En réalité la vraie catégorie est bien virginica. Il faudrait faire tourner l'algorithme k-NN en utilisant les 4 attributs pour calculer les distances et non pas 2 attributs.

EXERCICE 4

- 1) On doit convertir les données de sexe et fumeur en nombre. Par exemple masculin = 0 et féminin = 1, fumeur = 1, non fumeur = 0. Il faut ensuite créer un tableau $X = [s, a, m, f]$ comportant 100 lignes et un vecteur $Y = [p]$ qui contiendra les données de pression sanguine correspondante. On pourra alors appeler la fonction `model.fit(X, Y)`.
- 2) Si on choisit un modèle de régression linéaire multivariée, la pression sanguine sera donnée par la relation $p = c_0 + c_1 \times s + c_2 \times a + c_3 \times m + c_4 \times f$. Il y a donc 5 coefficients à déterminer.
- 3) On constate que le coefficient de corrélation est faible ce qui montre que l'approximation n'est pas pertinente. On pourrait essayer de lancer de nouvelles régressions en enlevant des données ou bien en créant d'autres en combinant les données. Par exemple en ajoutant un attribut correspondant à $s \times m$ on obtient un coefficient de corrélation de 0,53, ce est guère mieux.

EXERCICE 5

- 1) Il s'agit d'un problème de régression car les données de sortie recherchées sont continues.
- 2) Étant donné qu'il n'y a pas de couche cachée, on utilise uniquement les neurones de la couche de sortie. Il faut donc utiliser 3 neurones.
- 3) Comme il y a 6 attributs en entrée, il y aura 6×3 liens entre les données d'entrée et les 3 neurones soit 18 poids. Il faudra 3 biais car il y a 3 neurones.

4) Une fonction sigmoïde pour l'activation est pertinente pour obtenir un résultat entre 0 et 1 continu pour les sorties.

EXERCICE 6

1) On peut utiliser une régression logistique pour prédire le comportement car la sortie est binaire. On pourrait aussi utiliser un réseau de neurones.

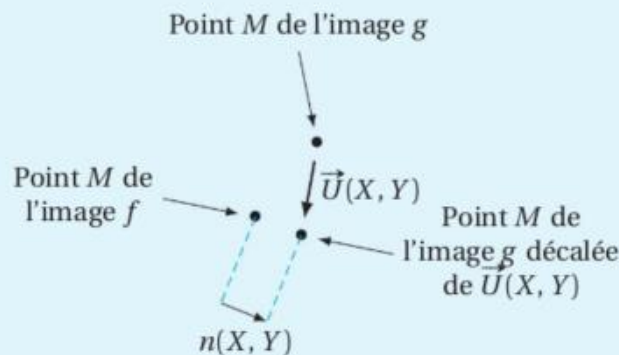
2) Pour analyser la pertinence de l'apprentissage mis en place, il faudrait extraire un certain nombre de données parmi les 350, par exemple 50 de manière aléatoire puis lancer l'apprentissage sur les 300 échantillons et analyser la prédiction sur les 50 échantillons test.

3) On constate que $13 + 32 = 45$ échantillons ont correctement été prédits. 6 échantillons ont été prédits comme bon, alors qu'en réalité ils étaient mauvais. Aucun échantillon a été prédit comme mauvais alors qu'il était bon. La justesse est calculée en faisant le rapport du nombre d'échantillons correctement triés sur le nombre d'échantillons analysés soit $45/50 = 0,9$. Cette justesse est particulièrement bonne pour cet exemple.

Corrigés des exercices d'approfondissement

EXERCICE A

1) L'erreur $n(X, Y) = g(X + U_X(X, Y), Y + U_Y(X, Y)) - f(X, Y)$ ce qui représente l'écart entre un point de l'image g décalée de $\vec{U}(X, Y)$ et le point correspondant de l'image de référence f . L'intégrale consiste à sommer tous ces écarts (au carré pour éliminer les problèmes de signes). Ainsi, la minimisation de cette fonction permet d'assurer que le déplacement trouvé est celui qui engendre le moins d'écart et donc celui qui est le plus proche du déplacement réel. C'est le principe de la méthode des moindres carrés.



► Figure 7.32. Principe de la minimisation de la fonction coût.

2) Le développement de Taylor est le suivant $g(X + U_X(X, Y), Y + U_Y(X, Y)) = g(X, Y) + \vec{\nabla} g(X, Y) \cdot \vec{U}(X, Y)$ au premier ordre.

Ainsi :

$$\begin{aligned} J(U_X(X, Y), U_Y(X, Y)) &= \iint (g(X + U_X(X, Y), Y + U_Y(X, Y)) - f(X, Y))^2 dX dY \\ &= \iint (g(X, Y) + \vec{\nabla} g(X, Y) \cdot \vec{U}(X, Y) - f(X, Y))^2 dX dY \\ &= \iint \left(g(X, Y) + \sum_{i=1}^6 \alpha_i \vec{\nabla} g(X, Y) \cdot \vec{\varphi}_i(X, Y) - f(X, Y) \right)^2 dX dY \end{aligned}$$

3) La minimisation consiste à calculer la dérivée partielle de la fonction coût par rapport à chaque variable (ici α_i) et chercher l'égalité à 0 de chaque dérivée (on obtient ainsi 6 équations). La dérivée partielle par rapport à α_i donne l'équation :

$$\frac{\partial J(\alpha_i)}{\partial \alpha_i} = 2 \iint \left(g(X, Y) + \sum_{j=1}^6 \alpha_j \vec{\nabla} g(X, Y) \cdot \vec{\varphi}_j(X, Y) - f(X, Y) \right) \times (\vec{\nabla} g(X, Y) \cdot \vec{\varphi}_i(X, Y)) dX dY = 0$$

4) On peut réécrire l'expression déterminée précédemment pour un i donné :

$$\begin{aligned} \iint \left(\sum_{j=1}^6 \alpha_j \vec{\nabla} g(X, Y) \cdot \vec{\varphi}_j(X, Y) \right) \times (\vec{\nabla} g(X, Y) \cdot \vec{\varphi}_i(X, Y)) dX dY = \\ \iint (f(X, Y) - g(X, Y)) \times (\vec{\nabla} g(X, Y) \cdot \vec{\varphi}_i(X, Y)) dX dY \end{aligned}$$

Le second membre est déjà de la forme demandée. Le membre de gauche correspond à la matrice. En inversant le signe somme et l'intégrale, on identifie que

$$R_{ij} = \iint \vec{\nabla} g(X, Y) \cdot \vec{\varphi}_j(X, Y) \times \vec{\nabla} g(X, Y) \cdot \vec{\varphi}_i(X, Y) dX dY$$

On obtient ainsi le système linéaire indiqué en prenant les 6 équations aux dérivées partielles. On remarque que la matrice R est symétrique.

5) Le vecteur gradient correspond au vecteur dont les composantes sont $\frac{\partial g(X, Y)}{\partial X}$ et $\frac{\partial g(X, Y)}{\partial Y}$. On peut donc approcher les dérivées par des dérivées à un pas en prenant des pas dX et dY .

$$\text{Ainsi, } \frac{\partial g(X, Y)}{\partial X} \approx \frac{g(X + dX, Y) - g(X, Y)}{dX} \text{ et } \frac{\partial g(X, Y)}{\partial Y} \approx \frac{g(X, Y + dY) - g(X, Y)}{dY}.$$

EXERCICE B

1) On utilise les commandes de lecture de fichiers `open` et `readlines` qui permet d'obtenir une liste de 2 chaînes de caractères pour chaque fichier. Il faut ensuite découper chaque ligne et remettre en forme le tableau X pour avoir n lignes et 2 colonnes. Le script est le même pour les deux fichiers.

```
with open("inputs.csv", "r") as f :
    datas=f.readlines()
    for i in range(len(datas)):
        datas[i]=datas[i].strip().split(";")
    n=len(datas[0])
    X=zeros((len(datas[0]),2))
    for i in range(n):
        X[i,:]=[float(datas[0][i]),float(datas[1][i])]
```

2) On cherche des valeurs quelconques pour les sorties à partir de données d'entrée (2 attributs différents) pour lesquels on connaît les valeurs. Il s'agit donc d'un problème d'apprentissage supervisé de type régression.

3) On utilise un modèle de régression linéaire multivariée car il y a 2 attributs en entrée. Compte-tenu de la position des points sur la figure, il est clair que l'on peut faire passer un plan d'équation $z = ax + by + c$ par ces points. L'approximation ainsi retenue est pertinente. La ligne `model.fit(X,Y[0])` permet donc de déterminer les meilleurs coefficients. On constate que l'erreur renvoyée par la ligne `model.score(X,Y[0])` de 0,994 montre que l'approximation est pertinente.

4) On doit définir 100×100 points entre les bornes indiquées. Pour cela, on peut utiliser la commande `numpy.linspace(a,b,n)` qui crée n points entre a et b .

```
Xp=[]
x=np.linspace(0,314,100)
y=np.linspace(576,1800,100)
for i in range(100):
    for j in range(100):
        Xp.append([x[i],y[j]])
```

5) Le modèle est pertinent car l'approximation passe par une grande majorité des points. L'équation numérique qui donne une valeur de couple notée `couple` en fonction du taux de carburant `carburant` et de la vitesse `vitesse` est la suivante : `couple=6.0 * carburant + 0.034 * vitesse -136.83`.

6) On constate que l'approximation obtenue par réseau de neurones passe par plus de points (justesse plus proche de 1) mais approche différemment la zone pour laquelle aucune donnée n'est disponible.

L'inconvénient est que le coût de calcul est beaucoup plus important avec cette méthode qu'avec la méthode de régression qui ne nécessitait que la détermination de 3 paramètres. Ici, étant donné qu'il y a 2 couches cachées à 100 et 50 neurones, qu'il y a 1 grandeur en sortie et 2 attributs en entrée, l'algorithme devra déterminer : 100×2 poids entre la couche d'entrée et la première couche, 100 biais pour la première couche, 100×50 poids entre la première et deuxième couche, 50 biais pour la deuxième couche et enfin 50×1 poids entre la deuxième couche et la dernière couche et 1 biais soit au total : 5250 poids et 151 biais ce qui est beaucoup plus coûteux que la méthode de régression linéaire.

EXERCICE C

1) On observe que l'allure de la courbe est une droite, on peut donc approcher la courbe par : $V = aT + b$. On peut relever $a \approx 1,75$ et $b \approx 76$.

2) On cherche à minimiser l'écart quadratique entre V_i et $aT_i + b$.

On définit alors la fonction coût : $J(a, b) = \sum_i (V_i - at_i - b)^2$ et le minimum est obtenu lorsque $\frac{\partial J(a, b)}{\partial a} = 0$ et $\frac{\partial J(a, b)}{\partial b} = 0$.

Ceci permet d'obtenir 2 équations $\sum_i t_i (V_i - at_i - b) = 0$ et $\sum_i (V_i - at_i - b) = 0$ soit un système linéaire en a et b qui peuvent être obtenus directement (ou par une méthode itérative).

3) Il faut bien choisir les bornes des vecteurs passés dans la fonction `linregress` pour faire l'approximation souhaitée.

```
def linregmobile(V,T,N):
    i=len(T)
    if len(V)>N:
        sol=linregress(T[i-N:i],V[i-N:i])
        vapp=sol[0]*t+sol[1]
    return vapp
```

4) On observe que l'erreur est limitée à quelques millilitres ce qui permet de conclure qu'on peut prendre $N = 300$ points pour la régression mobile.

EXERCICE D

- 1) Il y a 1797 images dans la librairie car cela correspond au nombre de lignes de data ou de target. Il y a 64 attributs (colonnes de data).
- 2) L'apprentissage sera de type supervisé car l'on connaît les lettres associées à chaque image. Étant donné qu'on veut déterminer un des 10 chiffres, on a affaire à un problème de classification. On peut utiliser un algorithme de type k-NN ou encore un réseau de neurones.
- 3) On remarque que les images ont une taille de $8 \times 8 = 64$. Ainsi les 64 attributs correspondent aux valeurs des pixels de chaque image.
- 4) On utilise l'algorithme k-NN en prenant $k = 5$. On limite le dataset aux 100 premières images uniquement. X correspond au dataset et y aux valeurs de sortie correspondantes aux échantillons du dataset. Après avoir défini le modèle, on lance la phase d'apprentissage (commande `fit`).
- 5) On constate que plus le nombre d'échantillons est important et plus la justesse est proche de 1, c'est à dire que les prédictions sont quasiment toutes bonnes. On remarque cependant qu'il n'est pas utile d'utiliser un nombre d'images trop important car la justesse n'évolue plus au bout d'un certain nombre d'échantillons. Par exemple, on peut se contenter de 200 images pour avoir de bons résultats.
- 6) On observe sur la matrice qu'en sommant les valeurs sur la deuxième ligne de la matrice (en partant du haut) qu'il y a 62 images représentant un 1.
Seules 52 ont correctement été identifiées, soit 84 % de réussite.
7 images ont été identifiées comme des 1 alors qu'elles correspondaient à d'autres chiffres (on somme les termes hors diagonale dans la deuxième colonne).
- 7) Sur les 606 images traitées :
 - avec $k = 2$, 548 ont été bien identifiées, soit 90 % de réussite
 - avec $k = 3$, 542 ont été bien identifiées, soit 89 % de réussite
 - avec $k = 4$, 541 ont été bien identifiées, soit 89 % de réussiteLes valeurs de k choisies dans cette simulation n'influencent pas le résultat.

EXERCICE E

- 1) Il s'agit d'un problème de classification multiclasse. On peut utiliser une méthode du type k-NN ou bien un réseau de neurones.
- 2) La matrice X contient les enregistrements en ligne (donc 24075 lignes) et les 60 caractéristiques en colonne (attributs). Ainsi la matrice a une taille de 24075×60 . Le vecteur y contient les classes associées à chaque enregistrement donc possède 24075 valeurs. Pour pouvoir travailler sur des nombres, on associe un nombre à chaque mouvement de 0 à 4 (indice dans la liste).
- 3) Il faut boucler sur chaque ligne lue en éliminant la première. Chaque ligne est ensuite découpée selon le «;». On sélectionne ensuite les colonnes 1 à 6 inclus et on convertit les chaînes de caractères en flottant.

```
with open("datas.csv","r") as f:
    datas=f.readlines()
X=[]
for i in range(1,len(datas)):
    datas[i]=datas[i].strip().split(";")
    x=[]
    for j in range(1,7):
        x.append(float(datas[i][j]))
    X.append(x)
```

- 4) On détermine de manière aléatoire nb indices compris entre 0 et le nombre maximum d'échantillons de X (environ 10 %). On sépare les données en 2 parties x_a, y_a pour réaliser l'apprentissage et x_t, y_t pour la phase d'inférence et vérifier la pertinence des algorithmes d'IA utilisés.
- 5) On constate que l'erreur est très proche de 1 ce qui laisse penser que la prédiction est très bonne. Ceci est confirmé par la matrice de confusion qui est quasiment diagonale. La justesse est égale à

$(580 + 587 + 484 + 368 + 243)/2281 = 0.991$. La majorité des prédictions qui ne sont pas correctes correspondent à des confusions entre le mouvement de course et de danse ce qui semble normal car les mouvements sont proches, ainsi que des confusions entre position debout et la marche. Il n'y a pas de confusion possible entre la position assise et debout ou un mouvement de marche (le smartphone étant dans la poche, on comprend que les mesures des accélérations sont complètement différentes en position assise ou debout par exemple).

6) Il y a $6 \times n_1$ poids entre la couche d'entrée et la couche cachée 1, $n_1 \times n_2$ entre les couches cachées et n_2 pour la couche de sortie soit $6n_1 + n_1 \times n_2 + n_2$ poids au total. Il faut également $n_1 + n_2 + 1$ biais.

7) On constate que plus on augmente le nombre de neurones par couche et plus l'erreur est proche de 1 mais le temps de calcul est forcément allongé car il y a plus d'inconnues à déterminer. Quand on observe la matrice de confusion on constate que la justesse est légèrement moins bonne 0,987 mais les résultats sont quand même globalement très satisfaisants.

Au final, on préférera la méthode k-NN qui est beaucoup plus rapide et qui donne de bons résultats.